🔴 Git Advanced Interview Mastery — A1 to A8

> **How to use this guide:** Every topic follows: concept → internals → short interview answer → deep dive → real-world example → interview Q&A → gotchas → connections. ⚠ marks high-frequency interview traps. Master these and you cover the top 20% of questions that filter senior/staff-level candidates.

A1. Git Internals Deep Dive ⚠

> The topics that separate senior engineers from everyone else. If you can explain the DAG, reflog, and partial clones fluently, you're in the top 5% of Git knowledge for interviews.

DAG (Directed Acyclic Graph) — How Git Models History

What it is (Simple Terms)

Git's commit history is not a simple list — it is a **Directed Acyclic Graph**. Each commit node points to one or more parent commits (directed), and you can never travel in a loop back to a previous commit (acyclic). This structure is what enables branches, merges, rebases, and all of Git's power.

Why It's a DAG and Not a Tree

A **tree** has exactly one parent per node. A DAG allows multiple parents. A merge commit has **two parents** — that's what makes it a DAG, not a tree. And it's acyclic because a commit cannot be its own ancestor — SHA hashes make this mathematically guaranteed.

```
Simple linear history (degenerate DAG — also a tree):
  C1 ← C2 ← C3 ← C4 (HEAD)

After a merge (true DAG — not a tree):
  C1 ← C2 ← C3 ← M  (merge commit, two parents)
              ↑   ↗
              F1 ← F2

After a rebase (linear DAG again):
  C1 ← C2 ← C3 ← F1' ← F2' (HEAD)
```

Key Properties of the Git DAG

1. Reachability A commit X is "reachable" from commit Y if you can walk parent pointers from Y and eventually reach X. This is how `git log A..B` works — it finds commits reachable from B but not from A.

```
# Commits in feature but NOT in main
git log main..feature

# Commits in either but NOT both (symmetric difference)
git log main...feature

# Check if commit abc1234 is an ancestor of HEAD
git merge-base --is-ancestor abc1234 HEAD
echo $?   # 0 = yes, 1 = no
```

2. Common Ancestor The **merge base** is the most recent common ancestor of two commits in the DAG. Git uses it for three-way merges and for `git rebase`.

```
# Find the common ancestor of two branches
git merge-base main feature/login
# Output: a3f9d2c4...  ← the SHA of the common ancestor commit

# Find all common ancestors (criss-cross merge scenarios)
git merge-base --all main feature/login
```

3. DAG Traversal Algorithms Git Uses

```
git log:          BFS/DFS from HEAD(s), topologically sorted by date
git merge-base:   Reverse BFS from both tips until a common node is found
git bisect:       Binary search on a linearized path through the DAG
git describe:     Walk backwards from HEAD to find the nearest tagged
commit
git pack-objects: Topological sort to compute deltas efficiently
```

The Commit Graph File (Performance Optimization)

For large repos with millions of commits, traversing the DAG by reading individual loose objects is slow. Git 2.18+ introduced a binary **commit-graph file** that pre-computes and caches:

- Parent relationships
- Generation numbers (topological depth from root)
- Root tree SHA for each commit

```
# Generate the commit graph
git commit-graph write --reachable

# Write with chain files (incremental updates for large repos)
git commit-graph write --reachable --split

# Verify the commit graph
git commit-graph verify
```

```
# Auto-write on fetch/push
git config --global fetch.writeCommitGraph true

# Check if it's being used
git config core.commitGraph
# true = using the graph file
```

Performance impact:

```
git log --oneline on linux kernel (1M+ commits):
  Without commit graph:  ~45 seconds
  With commit graph:     ~1.5 seconds  (30x faster)
```

Short Interview Answer

> "Git models commit history as a Directed Acyclic Graph — each commit points to one or more parent commits, enabling branches (diverging paths) and merges (converging paths). It's directed because parent pointers only go backwards in time, and acyclic because a commit cannot be its own ancestor — the SHA hash of a commit depends on its parent's SHA, making cycles mathematically impossible. Most Git operations — log, merge, rebase, bisect — are graph traversal algorithms walking this DAG."

How `git gc` and `git prune` Reclaim Space

The Lifecycle of Git Objects

```
1. You run git add/commit → loose objects created in
.git/objects/xx/yyyy...
2. Over time, thousands of loose objects accumulate
3. git gc runs automatically (or manually) → packs loose objects into
packfiles
4. Unreachable objects (orphaned commits from rebases, dropped stash
entries) → eventually pruned
```

`git gc` — What It Does Step by Step

```
# What git gc actually runs internally:
# 1. git pack-refs          → packs loose ref files into .git/packed-refs
# 2. git reflog expire      → removes reflog entries older than expiry
time
# 3. git pack-objects       → packs loose objects into a .pack file
# 4. git prune-packed       → removes loose objects already in packfiles
# 5. git prune              → removes unreachable objects
# 6. git worktree prune     → removes stale worktree state
# 7. git rerere gc          → cleans old rerere recordings
```

```
# Run manually
git gc

# Aggressive gc (deeper delta compression, much slower)
git gc --aggressive

# Dry run — see what would be pruned
git gc --dry-run

# gc with explicit prune time
git gc --prune=now           # prune everything immediately
git gc --prune=2.weeks.ago   # default: prune objects older than 2 weeks

# Check if gc is needed
git gc --auto                # auto-runs gc if thresholds are exceeded
```

The Loose Object Threshold

```
# Git auto-runs gc when loose objects exceed this threshold
git config gc.auto           # default: 6700 objects
git config gc.autoPackLimit  # default: 50 packfiles

# Disable auto gc (advanced — not recommended)
git config gc.auto 0
```

`git prune` — What Gets Pruned

`git prune` removes objects that are **unreachable from any ref** — branches, tags, stash, HEAD, ORIG_HEAD, FETCH_HEAD, and reflog entries.

```
# Prune unreachable objects older than 2 weeks
git prune

# Prune everything unreachable immediately
git prune --expire=now

# Dry run
git prune -n --expire=now

# Verbose output
git prune -v
```

What Protects Objects From Pruning

```
Objects are safe if reachable from any of:
  - Any branch (refs/heads/*)
  - Any tag (refs/tags/*)
  - HEAD and ORIG_HEAD
  - FETCH_HEAD, MERGE_HEAD
  - refs/stash
  - reflog entries (within expiry window, default 90 days)
  - CHERRY_PICK_HEAD, REBASE_HEAD
```

The Reflog Expiry Rules

```
# Reflog keeps objects alive for:
git config gc.reflogExpire           # 90 days for reachable commits
(default)
git config gc.reflogExpireUnreachable  # 30 days for unreachable commits
(default)

# Expire all reflog entries immediately
git reflog expire --expire=now --all

# After expiry, prune the now-unreachable objects
git prune --expire=now
```

`git count-objects` — Measuring Repo Size

```
git count-objects -v
# count:        23        ← loose objects
# size:         92        ← size in KB
# in-pack:      18291     ← objects in packfiles
# packs:        3         ← number of packfiles
# size-pack:    8420      ← size of packfiles in KB
# prune-packable: 0       ← loose objects already in packs (gc candidates)
# garbage:      0         ← objects in corrupt/invalid state
# size-garbage: 0
```

Packfile Delta Compression

When git packs objects, it finds similar objects and stores them as **deltas** (binary patches) against a base object. This is how a large repo with millions of versions of similar files stays manageable.

```
# Verify a packfile
git verify-pack -v .git/objects/pack/pack-abc123.idx | head -20
# SHA1 type size compressed-size [depth base-SHA]
# a3f9d2c blob 4096 182 0          ← base object
# b7e1234 blob 4102 14  1 a3f9d2c  ← delta of depth 1 against a3f9d2c
```

```
# Repack with maximum compression
git repack -a -d --depth=250 --window=250
# -a = into single packfile
# -d = delete redundant packs
# --depth = max delta chain depth
# --window = how many objects to compare for delta
```

Reflog — The Complete Safety Net ⚠

What it is (Simple Terms)

The reflog is a **local journal of every position HEAD and branch refs have ever pointed to** on your machine. It is your complete undo history — even for operations that rewrote history. If you can name it, reflog can find it.

How Reflog Works Internally

Every time HEAD moves — commit, checkout, reset, rebase, cherry-pick, merge — Git appends an entry to `.git/logs/HEAD`. Branch refs also have their own logs in `.git/logs/refs/heads/`.

```
cat .git/logs/HEAD
# 0000000 a3f9d2c Jane Smith <jane@co.com> 1700000000 +0530 commit
(initial): Initial commit
# a3f9d2c b7e1234 Jane Smith <jane@co.com> 1700000100 +0530 commit: feat:
add auth
# b7e1234 c4d5678 Jane Smith <jane@co.com> 1700000200 +0530 checkout:
moving from main to feature/login
# c4d5678 d5e6789 Jane Smith <jane@co.com> 1700000300 +0530 rebase
(finish): returning to refs/heads/feature/login
```

Reading the Reflog

```
# Show HEAD reflog (most recent first)
git reflog
# HEAD@{0}: rebase (finish): returning to refs/heads/feature/login
# HEAD@{1}: rebase (pick): feat: add login form
# HEAD@{2}: rebase (start): checkout main
# HEAD@{3}: commit: WIP: half-done auth changes
# HEAD@{4}: checkout: moving from main to feature/login
# HEAD@{5}: reset: moving to HEAD~3
# HEAD@{6}: commit: feat: add OAuth button

# Show reflog for a specific branch
git reflog show main
git reflog show feature/login

# Show with timestamps
```

```
git reflog --date=iso
git reflog --date=relative

# Limit entries
git reflog -10
```

Reflog Syntax

```
# HEAD@{N} = HEAD's position N moves ago
git show HEAD@{0}     # current position (same as HEAD)
git show HEAD@{1}     # one move ago
git show HEAD@{5}     # five moves ago

# Branch@{N} = branch's position N moves ago
git show main@{1}

# Time-based syntax
git show HEAD@{2.hours.ago}
git show main@{yesterday}
git show HEAD@{"2024-01-15 10:30:00"}
```

Recovery Scenarios with Reflog

Scenario 1: Accidental `git reset --hard`

```
# You ran git reset --hard HEAD~5 and lost 5 commits
git reflog
# HEAD@{0}: reset: moving to HEAD~5
# HEAD@{1}: commit: feat: add payment gateway ← this is what you lost
# HEAD@{2}: commit: feat: add cart persistence
# ... (3 more commits)

# Recover — reset back to what HEAD was before the reset
git reset --hard HEAD@{1}
# or: git reset --hard <sha-from-reflog>
```

Scenario 2: Deleted a branch

```
# You deleted feature/login and it wasn't merged
git branch -D feature/login

# Find where feature/login was pointing before deletion
git reflog show feature/login
# feature/login@{0}: branch: Created from HEAD
# feature/login@{1}: commit: feat: add OAuth

# OR: search HEAD reflog for the branch name
```

```
git reflog | grep "feature/login"

# Recreate the branch at the correct SHA
git checkout -b feature/login d5e6789
```

Scenario 3: Recovering commits after a bad rebase

```
# Interactive rebase accidentally dropped some commits
git reflog
# HEAD@{0}: rebase (finish): returning to feature/login
# HEAD@{1}: rebase (start): checkout main
# HEAD@{4}: commit: feat: the commit I dropped

# The dropped commits are in the reflog as orphans
git cherry-pick HEAD@{4}
# or: git reset --hard HEAD@{4} to go back before the rebase
```

Scenario 4: Recovering a stash you dropped

```
# git stash drop removed a stash
# Stash entries are commits — find them in reflog
git reflog stash
# stash@{0}: WIP on main: a3f9d2c feat: add auth

# If already dropped, search the object store
git fsck --unreachable | grep commit | cut -d' ' -f3 | xargs git log --
oneline --no-walk
# Find the stash commit SHA and cherry-pick it
```

Scenario 5: Undo a revert

```
# You ran git revert HEAD and now regret it
git reflog
# HEAD@{0}: revert: Revert "feat: add payment gateway"
# HEAD@{1}: commit: feat: add payment gateway

git reset --hard HEAD@{1}
```

Reflog Expiry and Limits

```
# Default expiry: 90 days for reachable commits, 30 days for unreachable
git config --global gc.reflogExpire 90.days
git config --global gc.reflogExpireUnreachable 30.days

# Delete specific reflog entry
```

```
git reflog delete HEAD@{5}

# Expire all old entries
git reflog expire --expire=30.days.ago --all

# Reflog is LOCAL — not pushed to remote, not shared with teammates
```

Short Interview Answer

> "The reflog is Git's local safety net — it records every position HEAD and branch refs have ever pointed to on your machine, with timestamps and operation descriptions. It's the answer to almost every 'I lost my work' scenario: accidental reset --hard, deleted branches, dropped stash entries, and bad rebases. I use `git reflog` to find the SHA of the lost commit, then `git reset --hard <sha>` or `git checkout -b recover <sha>` to get it back. The reflog retains entries for 90 days by default. It's local only — not synced to remotes."

⚠ Gotchas

- Reflog is **per-machine**. If your teammate deleted a branch on their machine, their reflog won't help you. Only the reflog on the machine that had that branch is useful.
- `git gc --prune=now` followed by `git reflog expire --expire=now --all` will permanently delete orphaned objects even if they're in the reflog. This is irreversible.
- Bare repositories (used on Git servers) have reflog disabled by default.
- GitHub/GitLab do not expose server-side reflogs to users.

Partial Clones and Sparse Checkouts for Large Repos

The Problem

A repo with 10 years of history, thousands of contributors, and large binary assets can be 50GB+. A full `git clone` downloads every version of every file ever committed. For CI agents, new contributors, and monorepos, this is unacceptably slow.

Partial Clone (`--filter`)

Partial clones download commits and trees but **defer downloading blob content** until it's actually needed. Git 2.19+.

```
# Blobless clone: downloads all commits and trees, defers blobs
# Best balance of functionality and speed
git clone --filter=blob:none https://github.com/org/repo.git

# Treeless clone: only downloads commits, defers trees and blobs
# Fastest clone, but many operations require network fetches
git clone --filter=tree:0 https://github.com/org/repo.git

# No blobs larger than 1MB
git clone --filter=blob:limit=1m https://github.com/org/repo.git
```

```
# After blobless clone, missing blobs are fetched on demand
cd repo
git checkout main    # fetches needed blobs automatically
cat src/bigfile.bin  # fetches the blob for this specific file
```

What each filter defers:

Filter	Commits	Trees	Blobs	Use Case
`blob:none`	✅ fetched	✅ fetched	❌ deferred	Code browsing, CI
`tree:0`	✅ fetched	❌ deferred	❌ deferred	Shallow history inspection
`blob:limit=Xm`	✅ fetched	✅ fetched	Large only deferred	Mixed code + assets repos

Sparse Checkout

Sparse checkout lets you check out **only a subset of the working tree** — specific directories instead of the full monorepo.

```
# --- Cone mode (recommended, fast) ---
# Initialize sparse checkout
git clone --no-checkout https://github.com/org/monorepo.git
cd monorepo
git sparse-checkout init --cone

# Add directories you want
git sparse-checkout set services/payments services/auth shared/utils
git checkout main

# Add more directories later
git sparse-checkout add services/notifications

# See what's currently checked out
git sparse-checkout list

# Disable sparse checkout (full tree)
git sparse-checkout disable

# --- Non-cone mode (pattern-based, slower) ---
git sparse-checkout init
echo "services/payments/**" >> .git/info/sparse-checkout
git checkout main
```

Combining Partial Clone + Sparse Checkout (The Power Combo)

```
# Best practice for monorepos: partial clone + sparse checkout
git clone --filter=blob:none --no-checkout
```

```
https://github.com/org/monorepo.git
cd monorepo
git sparse-checkout init --cone
git sparse-checkout set services/payments
git checkout main

# Effect:
# - Only commits + trees downloaded (blobless)
# - Only services/payments tree checked out
# - Clone time: seconds instead of minutes
# - Disk usage: MB instead of GB
```

Real-World CI/CD Usage

```
# GitHub Actions: optimized clone for CI
- name: Checkout (optimized)
  uses: actions/checkout@v3
  with:
    filter: blob:none          # blobless clone
    sparse-checkout: |         # only checkout relevant paths
      services/payments
      shared/utils
    fetch-depth: 0             # full history for git describe
```

Git Protocol Internals — Smart HTTP, SSH, Git Protocol

The Four Git Protocols

```
1. Local       file:///path/to/repo.git    → direct filesystem access
2. Git         git://host/repo.git         → Git daemon, port 9418, no
auth
3. SSH         git@github.com:org/repo.git → authenticated, encrypted
4. HTTP/HTTPS  https://github.com/org/repo → firewall-friendly, widely
supported
```

Smart HTTP vs Dumb HTTP

Dumb HTTP serves Git repos as static files. No server-side computation — the client does all the work by fetching individual files. Inefficient, rarely used today.

Smart HTTP (used by GitHub, GitLab, Bitbucket) involves the server understanding Git and negotiating efficient packfile transfers.

```
Smart HTTP clone (simplified):
  Client → GET /repo.git/info/refs?service=git-upload-pack
```

```
  Server → returns list of refs and capabilities
  Client → POST /repo.git/git-upload-pack (wants: SHA1 SHA2..., have: SHA3
SHA4...)
  Server → sends packed objects (packfile)
  Client → unpacks objects into .git/objects/
```

The Pack Protocol (Fetch Negotiation)

The most important internal protocol is the **pack negotiation** that happens during `git fetch` and `git clone`.

```
1. Reference Discovery:
   Server sends: all branch/tag SHAs
   Client receives: "here's what I have"

2. Want/Have Negotiation:
   Client says: "I want SHA_A, SHA_B, SHA_C"
   Client says: "I have SHA_X, SHA_Y, SHA_Z" (from previous fetches)
   Server computes: what objects does client need that aren't covered by
"have" list?

3. Packfile Transfer:
   Server sends: a packfile containing exactly the needed objects
   Client unpacks into .git/objects/

4. Reference Update:
   Client updates remote-tracking refs (origin/main, etc.)
```

SSH Protocol

```
# SSH URL format
git@github.com:org/repo.git    # user@host:path

# What happens:
# 1. SSH connection established to github.com as user "git"
# 2. SSH key authentication
# 3. Server runs: git-upload-pack /org/repo.git (for fetch)
#                 git-receive-pack /org/repo.git (for push)
# 4. Pack protocol runs over the SSH channel

# Test SSH connectivity
ssh -T git@github.com
# Hi username! You've successfully authenticated.

# Debug SSH connection
ssh -vT git@github.com 2>&1 | head -30
GIT_SSH_COMMAND="ssh -v" git fetch origin
```

Wire Protocol v2 (Git 2.26+)

Git protocol v2 significantly reduces the reference advertisement phase for large repos (those with thousands of branches/tags).

```
# Check protocol version
git config protocol.version
# 2 (default since Git 2.26)

# Force protocol v2
git -c protocol.version=2 fetch origin

# What v2 improves:
# - Ref filtering: only advertise refs the client wants
# - Capability advertisement: once per session, not per request
# - Significant speedup for repos with thousands of refs
```

How Git Ensures Data Integrity End-to-End

The SHA-1 Chain of Trust

Every Git object's SHA is computed from its **complete content** including pointers to parents and trees. This creates an **unbreakable chain**:

```
Blob SHA     = SHA1("blob" + size + content)
Tree SHA     = SHA1("tree" + size + [mode + filename + blob/tree SHA]...)
Commit SHA   = SHA1("commit" + size + tree-SHA + parent-SHA + author +
message)

If ANY byte changes in any file → blob SHA changes
→ tree SHA changes (references the blob SHA)
→ commit SHA changes (references the tree SHA)
→ all descendant commit SHAs change

This means: a single bit flip anywhere in history changes every commit SHA
from that point forward.
```

Verification Commands

```
# Verify all objects in the repo
git fsck
# Checking object connectivity and validity.
# Checking connectivity: done.

# Verbose fsck
git fsck --full --strict
```

```
# Verify a specific packfile
git verify-pack -v .git/objects/pack/pack-abc123.idx

# Check that a commit's hash matches its content
git cat-file commit HEAD | git hash-object --stdin -t commit
# Should match: git rev-parse HEAD

# Verify a remote's objects match yours
git fetch --dry-run   # checks that received objects are valid
```

Transfer Integrity

```
# Git validates SHA on every received object during fetch/clone
# If an object is corrupted in transit, the SHA won't match → rejected
# This is automatic and cannot be disabled

# HTTPS adds TLS for transport encryption
# SSH adds encryption + authentication

# GitHub/GitLab additional hardening:
# - Refuse pushes containing known SHA-1 collision attacks (ShaAttered
protection)
# - Signed commits for verified identity
# - Protected branches prevent force-pushes that could introduce
collisions
```

SHA-1 Collision Vulnerability and SHA-256 Migration

SHA-1 has known theoretical collision vulnerabilities (SHAttered attack, 2017). Git is migrating to SHA-256.

```
# Create a new repo with SHA-256 object format (Git 2.29+)
git init --object-format=sha256

# Check format of existing repo
git rev-parse --show-object-format
# sha1 (most repos today)
# sha256 (new repos on Git 2.29+)

# GitHub/GitLab protect against SHA-1 collisions server-side
# Git itself has collision detection since 2.13
git config transfer.fsckObjects true    # verify all received objects
git config receive.fsckObjects true     # verify on push
git config fetch.fsckObjects true       # verify on fetch
```

Short Interview Answer (Full A1)

> "Git models history as a DAG where commits point to parents, enabling branches and merges. The SHA-1 hash chain means any modification to history changes every subsequent commit's SHA — this is Git's end-to-end integrity guarantee. `git gc` packs loose objects into delta-compressed packfiles and prunes unreachable objects, but objects referenced by reflog are protected for 90 days. The reflog is the complete local undo history — every HEAD movement is recorded, enabling recovery from almost any mistake. For large repos, partial clones with `--filter=blob:none` defer blob downloads, and sparse checkout reduces the working tree to only needed directories."

A2. Advanced Rebase Patterns ⚠

`git rebase --onto` — Transplanting Commit Ranges ⚠

What it is (Simple Terms)

`git rebase --onto` lets you **move a specific range of commits** to a completely new base, independently of where the branch is currently rooted. It's the surgical tool of Git history rewriting.

The Three-Argument Form

```
git rebase --onto <newbase> <upstream> <branch>

# Meaning:
# Take commits that are in <branch> but NOT in <upstream>
# and replay them on top of <newbase>
```

Scenario 1 — Feature Branch Accidentally Branched from Wrong Branch

```
Problem:
  main:       C1 → C2 → C3
  develop:    C1 → C2 → D1 → D2
  feature:    C1 → C2 → D1 → D2 → F1 → F2
                                  (should have branched from main, not
develop)

Goal: Move F1 and F2 onto main, discarding D1 and D2

Command:
  git rebase --onto main develop feature

Explanation:
  --onto main    → new base is main
  develop        → exclude commits reachable from develop (excludes D1,
D2)
  feature        → the branch to transplant
```

```
Result:
  main:       C1 → C2 → C3
  feature:    C1 → C2 → C3 → F1' → F2' (F1,F2 replayed on C3)
```

```
# Practical example
git log --oneline develop
# d2 commit: update config for dev environment
# d1 commit: add dev-only logging

git log --oneline feature
# f2 feat: add payment UI
# f1 feat: payment service skeleton
# d2 commit: update config for dev environment  ← don't want these
# d1 commit: add dev-only logging               ← don't want these

git rebase --onto main develop feature
# F1 and F2 replayed onto main, D1 and D2 discarded
```

Scenario 2 — Remove Commits from the Middle of History

```
Before:
  main: C1 → C2 → C3 → C4 → C5
  (C3 was a bad commit — experimental code never meant for main)

Goal: Remove C3, keep C4 and C5

Command:
  git rebase --onto C2 C3 main

Explanation:
  --onto C2   → new base is C2
  C3          → exclude C3 and everything it can reach below C4
  main        → branch to rewrite

Result:
  main: C1 → C2 → C4' → C5'  (C3 is gone)
```

```
# Remove a specific commit range from a branch
git log --oneline main
# c5 feat: final polish
# c4 feat: add tests
# c3 fix: experimental approach (remove this)
# c2 feat: add auth
# c1 initial commit

git rebase --onto c2_sha c3_sha main
# or using relative refs:
```

```
# Find c3's parent first
git rebase --onto HEAD~3 HEAD~2 main
```

Scenario 3 — Transplant a Subtopic Branch

```
Before:
  main:     A → B → C
  server:   A → B → C → D → E
  client:   A → B → C → D → E → F → G
  (client branched from server, but F and G are client-only features)

Goal: Rebase F and G directly onto main

Command:
  git rebase --onto main server client

Result:
  main:   A → B → C
  client: A → B → C → F' → G'  (D and E excluded)
```

Scenario 4 — Two-Argument Form (Simpler Case)

```
# Two-argument form: moves the current branch onto newbase
git rebase --onto main feature/login
# Equivalent to: git rebase --onto main $(git merge-base HEAD main)

# This is useful when you want to rebase onto a specific commit
git rebase --onto abc1234 HEAD~3
# Take last 3 commits and put them on abc1234
```

⚠ Gotchas

- After `--onto`, you typically need `git push --force-with-lease` if the branch was already pushed.
- SHA references in the three-argument form are **exclusive** on the upstream end — `git rebase --onto newbase A B` replays commits **after A** up to and including B.
- When moving commits that depend on context from excluded commits, you'll get conflicts.

Rebase with `--autosquash` and `fixup!` Commits

What it is

`--autosquash` automatically reorganizes commits during an interactive rebase based on special commit message prefixes: `fixup!` and `squash!`.

The Workflow

```
# Working on a feature, you make a commit
git commit -m "feat: add OAuth login"

# Later you find a bug in that commit
git add src/oauth.js
git commit -m "fixup! feat: add OAuth login"
# Note: message starts with "fixup!" followed by EXACT message of target
commit

# Run interactive rebase with autosquash
git rebase -i --autosquash HEAD~5

# Git AUTOMATICALLY reorganizes the editor to:
pick a3f9d2c feat: add OAuth login
fixup b7e1234 fixup! feat: add OAuth login   ← moved here automatically
pick c4d5678 feat: add cart persistence

# Without autosquash you'd have to manually move and relabel
```

`fixup!` vs `squash!`

```
# fixup! → absorbs the commit into the target, DISCARDS the fixup's
message
git commit -m "fixup! feat: add OAuth login"

# squash! → absorbs the commit, KEEPS the squash commit's message (prompts
you)
git commit -m "squash! feat: add OAuth login"
```

Making `--autosquash` the Default

```
git config --global rebase.autoSquash true
# Now git rebase -i always behaves as if --autosquash was passed
```

The `--fixup` Flag Shortcut

```
# Instead of typing the full "fixup! <original message>":
git commit --fixup HEAD        # fixup for HEAD commit
git commit --fixup a3f9d2c    # fixup for a specific SHA

# Git automatically creates the "fixup! <message>" format
git commit --squash a3f9d2c   # creates "squash! <message>"
```

Real-World Team Workflow

```
# On a PR with multiple review iterations:

# Initial work
git commit -m "feat: add OAuth login"
git commit -m "feat: add session management"

# Reviewer feedback → create fixup commits
git add .
git commit --fixup HEAD~1    # fix for OAuth login
git add .
git commit --fixup HEAD      # fix for session management

# Before merging: clean up with autosquash
git rebase -i --autosquash origin/main

# Result: clean 2 commits, review fixes absorbed invisibly
```

Splitting a Commit During Interactive Rebase

When You Need This

You made one large commit that contains logically separate changes. PR reviewers want them split for clarity.

Step-by-Step Process

```
# Current history
git log --oneline
# a3f9d2c feat: add auth AND payment service (should be two commits)
# b7e1234 fix: previous thing

# Start interactive rebase
git rebase -i HEAD~2

# In the editor, mark the commit as 'edit'
edit a3f9d2c feat: add auth AND payment service
pick b7e1234 fix: previous thing
# Save and close

# Rebase pauses at the commit marked 'edit'
# HEAD is now AT that commit (the commit exists but we can amend it)

# Undo the commit but keep changes staged
git reset HEAD^
# or: git reset --mixed HEAD^ (same thing, unstages everything)

# Verify what's in working directory
git status
# Changes not staged for commit:
```

```
#   modified: src/auth/oauth.js
#   modified: src/payments/gateway.js

# Commit the first logical chunk
git add src/auth/
git commit -m "feat: add OAuth authentication"

# Commit the second logical chunk
git add src/payments/
git commit -m "feat: add payment gateway integration"

# Continue the rebase
git rebase --continue

# Result: the original one commit is now two commits
git log --oneline
# new2 feat: add payment gateway integration
# new1 feat: add OAuth authentication
# b7e1234 fix: previous thing
```

Splitting with `git add -p`

```
# If changes are mixed within the SAME file:
git reset HEAD^
git add -p src/mixed-file.js   # interactively stage specific hunks
git commit -m "feat: first logical change"
git add src/mixed-file.js      # stage the rest
git commit -m "feat: second logical change"
git rebase --continue
```

Rebasing a Long-Lived Feature Branch Safely Without Conflicts

The Challenge

A feature branch that's been in development for 3 months while `main` has received 200 commits will have accumulated conflicts in layers. A naive rebase will hit conflict after conflict.

Strategy 1 — Incremental Rebase (Recommended)

Instead of one massive rebase at the end, rebase frequently in small increments:

```
# Every morning, rebase onto latest main
git fetch origin
git rebase origin/main

# Resolve any conflicts (few, since main moved little overnight)
# Continue working
```

```
# 3 months later: branch is always current, final rebase is trivial
```

Strategy 2 — Staged Rebase for Large Divergence

```
# Step 1: Find the merge-base (where you branched off)
git merge-base HEAD origin/main
# → ancestor_sha

# Step 2: Rebase first onto an intermediate point
# (e.g., to where main was 1 month ago, using date)
git log --oneline --after="1.month.ago" origin/main | tail -1
# → mid_sha

git rebase --onto mid_sha ancestor_sha feature/big-feature
# Resolve conflicts here (should be fewer)

# Step 3: Rebase from mid_sha to current main
git rebase origin/main
# Resolve remaining conflicts (also fewer)
```

Strategy 3 — `git rerere` as a Conflict Memory

```
# Enable rerere BEFORE you start rebasing
git config rerere.enabled true

# First rebase attempt: resolve conflicts manually
git rebase origin/main
# ... resolve conflict in src/config.js: accept 'ours' version ...
git add src/config.js
git rebase --continue

# rerere records the resolution

# Next time you rebase (after reverting and trying again, or in future
rebases):
git rebase origin/main
# Git automatically applies the remembered resolution for src/config.js
# "Resolved 'src/config.js' using previous resolution."
```

Strategy 4 — Merge Instead for Very Long-Lived Branches

```
# If a branch has lived for 6+ months, a rebase might be impractical
# Use a merge instead (preserves history, one conflict resolution per
merge point)
git checkout feature/big-feature
```

```
git merge origin/main   # resolve conflicts once
git push origin feature/big-feature

# Open the PR as a merge — use "merge commit" strategy
# The PR history will show the full parallel development
```

`--update-refs` — Rebasing Stacked Branches

```
# If you have stacked branches:
# main → feature/base → feature/extension

# Rebasing feature/extension onto new main would normally
# NOT update feature/base pointer
# Git 2.38+ --update-refs fixes this:

git rebase --update-refs origin/main
# Automatically updates ALL intermediate branches that were moved
```

Rebase in a Team Context — The Golden Rule and When to Break It

The Golden Rule

> **Never rebase commits that have already been pushed to a shared branch.**

The reason: rebasing creates new SHAs. Any teammate who has fetched the old SHAs now has a diverged history. They must `git pull --rebase` or `git reset --hard origin/branch` to recover — and they lose any work they built on top of the old SHAs without warning.

When It's Acceptable to Break the Rule

```
1. Feature branch only you use:
   Your personal feature branch, no teammates have cloned it.
   → Safe to rebase and force push with --force-with-lease

2. Explicitly coordinated team rebase:
   Team agrees: "we're all going to git reset --hard origin/feature after
Jane's rebase"
   → Painful but feasible for small teams

3. Protected branches with merge queues:
   GitHub Merge Queue rebases PRs automatically before merging.
   This is "safe" because the rebase happens atomically with the merge.

4. Short-lived PR branches:
   A PR branch lives for hours to a few days.
   If you rebase before anyone reviews, it's generally safe.
   If someone has already reviewed and left comments tied to specific SHAs
→ be careful.
```

The Workflow That Makes Rebase Always Safe

```
# Rule: never push a branch more than once without coordinating
# Workflow for PR-based development:

# 1. Create branch, work
git checkout -b feature/oauth
# ... commits ...

# 2. First push: opens the PR
git push -u origin feature/oauth

# 3. Before any review, cleanup is fine
git rebase -i --autosquash origin/main   # clean up WIP commits
git push --force-with-lease              # safe: no one has reviewed yet

# 4. After reviews start: NO more rebase
# Address feedback with new commits, do NOT rebase
# Reviewers' comments are tied to specific SHAs — rebase breaks GitHub's
diff view

# 5. Squash at merge time (not rebase):
# Use GitHub's "Squash and Merge" button instead
# → clean history without the teammate disruption
```

Short Interview Answer

> "`git rebase --onto` is the surgical tool for transplanting commits — it lets you move a commit range to a completely new base, separate from where it was anchored. I use it when a feature branch was accidentally created from the wrong branch, or to remove intermediate commits. `--autosquash` automates PR cleanup by recognizing `fixup!` prefixed commits and automatically squashing them during interactive rebase. The golden rule for team rebase is: never rebase what's already shared, because it changes SHAs and breaks your teammates' history. The pragmatic exception is your own feature branch before any review has started."

A3. Large Repository Strategies

Git LFS (Large File Storage) — How It Works Internally

What it is (Simple Terms)

Git LFS replaces large files in your repository with small **pointer files** containing a hash. The actual file content is stored on a separate LFS server. This keeps the Git repository fast while still tracking large binary assets under version control.

Why It Exists

Without LFS, every version of every large binary (PSD, MP4, model weights, DLL) is stored as a blob in `.git/objects/`. A 500MB video committed 20 times → 10GB in the repo. `git clone` downloads all of it. With LFS, the clone downloads only pointer files (a few bytes each) and fetches actual content on demand.

How LFS Works Internally

```
# Without LFS — what Git stores for a 100MB file:
.git/objects/ab/cdef1234... ← 100MB blob stored directly in object store

# With LFS — what Git stores:
.git/objects/ab/cdef1234... ← tiny pointer file (130 bytes):
version https://git-lfs.github.com/spec/v1
oid
sha256:9f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08
size 104857600

# Actual file stored at: https://your-lfs-server/objects/9f86d...
```

Installing and Using Git LFS

```
# Install LFS (one-time, system-wide)
git lfs install

# Track specific file types
git lfs track "*.psd"
git lfs track "*.mp4"
git lfs track "models/**"

# What this creates in .gitattributes:
cat .gitattributes
# *.psd filter=lfs diff=lfs merge=lfs -text
# *.mp4 filter=lfs diff=lfs merge=lfs -text
# models/** filter=lfs diff=lfs merge=lfs -text

# Commit .gitattributes
git add .gitattributes
git commit -m "chore: configure LFS for binary assets"

# From now on, adding tracked files automatically uses LFS
git add design/mockup.psd   # stored in LFS, pointer in repo
git commit -m "design: add initial mockup"
git push   # pushes pointer to Git, uploads actual file to LFS server
```

LFS in CI/CD Pipelines

```
# Clone with LFS content
git clone --recurse-submodules https://repo.git
# LFS files are automatically downloaded if lfs is installed

# Skip LFS download (for CI that doesn't need binary assets)
GIT_LFS_SKIP_SMUDGE=1 git clone https://repo.git
# Pointer files are checked out instead of actual content

# Download specific LFS files
git lfs pull --include="models/production/**"

# GitHub Actions
- uses: actions/checkout@v3
  with:
    lfs: true   # automatically fetches LFS objects
```

⚠ LFS Gotchas

```
1. LFS storage costs money separately on GitHub/GitLab
2. git clone --depth=1 still downloads LFS objects — use
GIT_LFS_SKIP_SMUDGE to skip
3. Forking a repo does NOT fork the LFS objects by default (GitHub-
specific)
4. git diff on LFS files shows pointer diffs, not content diffs
5. LFS server must be available during push/pull — offline work is limited
6. Migrating existing large files to LFS requires git lfs migrate
(rewrites history)
```

```
# Migrate existing large files to LFS (rewrites history)
git lfs migrate import --include="*.psd" --everything
# This rewrites ALL commits — treat like git filter-repo (need to force
push)
```

Sparse Checkout (`--sparse`, `--filter=blob:none`)

Cone Mode vs No-Cone Mode

```
# Cone mode (default since Git 2.37):
# Fast, simple — you specify directories, Git includes all files in those
dirs
git sparse-checkout init --cone
git sparse-checkout set src/payments tests/

# Internally stores patterns as directory set (faster pattern matching)
cat .git/info/sparse-checkout
```

```
# /*
# !/*/
# /src/payments/
# /tests/

# No-cone mode:
# Flexible — supports full gitignore-style patterns
# But significantly slower for large repos
git sparse-checkout init
git sparse-checkout set --no-cone "src/payments/**"
"!src/payments/legacy/**"
```

Checking Sparse Status

```
# See what's currently checked out
git sparse-checkout list

# Check if sparse checkout is enabled
git config core.sparseCheckout
# true or false

# Add more paths without replacing existing
git sparse-checkout add src/notifications

# Disable (restore full tree)
git sparse-checkout disable
```

Sparse Checkout in CI Monorepo Pipelines

```
#!/bin/bash
# CI script: only checkout the service being built
SERVICE=$1   # e.g., "payments", "auth", "notifications"

git clone --filter=blob:none --no-checkout
https://github.com/org/monorepo.git
cd monorepo
git sparse-checkout init --cone
git sparse-checkout set "services/$SERVICE" "shared/utils" "shared/types"
git checkout main

echo "Checked out services/$SERVICE with shared dependencies"
du -sh .   # Show disk usage (significantly reduced)
```

Shallow Clones in CI (`--depth=1`) and Their Gotchas ⚠

Why Shallow Clones Are Used in CI

```
# Full clone of a large repo: 5+ minutes
git clone https://github.com/org/large-repo.git

# Shallow clone (last 1 commit): < 5 seconds
git clone --depth=1 https://github.com/org/large-repo.git

# CI doesn't need full history for:
# - Build and test
# - Static analysis
# - Deployment
```

The Gotchas in Detail

Gotcha 1: `git describe` fails

```
# git describe needs to find the nearest tag by walking history
git describe
# fatal: No names found, cannot describe anything.

# Fix: fetch tags
git fetch --tags --unshallow
# or in GitHub Actions:
# fetch-depth: 0
```

Gotcha 2: Merge base not found

```
# git merge-base fails when the common ancestor isn't in the shallow
history
git merge-base HEAD origin/main
# fatal: Not a valid object name origin/main

# Fix: deepen the clone
git fetch --deepen=50 origin main
# or fully unshallow:
git fetch --unshallow
```

Gotcha 3: History-dependent scripts fail

```
# Scripts that count commits, generate changelogs, or check blame
# all fail with shallow clones

# Count commits since v2.0.0
git rev-list v2.0.0..HEAD --count
# fatal: no such path 'v2.0.0' in HEAD
```

```
# Fix: fetch the specific tag
git fetch origin tag v2.0.0 --no-tags
```

Gotcha 4: `git log` shows incomplete history

```
# git log only shows commits within the depth window
git log --oneline | wc -l
# 1  (only the single shallow commit)

# Fix: unshallow when needed
git fetch --unshallow
```

Gotcha 5: Submodules in shallow repos

```
# Shallow clone + submodules = double gotcha
git clone --depth=1 --recurse-submodules https://repo.git
# May fail if submodule's SHA isn't at the tip of the fetched history

# Fix:
git clone --depth=1 https://repo.git
cd repo
git submodule update --init --depth=1  # shallow submodules too
```

The Recommended CI Pattern

```
# GitHub Actions: balanced approach
- uses: actions/checkout@v3
  with:
    fetch-depth: 0   # Full history for git describe and changelog
    # OR:
    fetch-depth: 1   # Shallow for pure build/test (add fetch steps if
needed)

# For monorepo with path-based triggers:
- uses: actions/checkout@v3
  with:
    filter: blob:none  # blobless, not shallow — gets full history without
blob content
    fetch-depth: 0
```

Scalar and VFS for Git — Microsoft's Approach to Monorepos

The Problem at Microsoft Scale

The Windows codebase: 3.5 million files, 300GB of history, 4,000 engineers. Standard Git operations took minutes or crashed entirely.

VFS for Git (GVFS) — Virtual Filesystem

Microsoft built a **virtual filesystem layer** that makes the working directory appear to contain all files, but only actually downloads them when accessed.

```
Without VFS for Git: git checkout downloads ALL 3.5M files → takes hours
With VFS for Git: git checkout "downloads" 3.5M file stubs instantly
                  Reading a file → stub intercepts → downloads actual
content → returns it
```

VFS for Git is Windows-specific (uses Windows ProjFS). It has been largely superseded by Scalar.

Scalar — Cross-Platform Monorepo Optimizer

Scalar is built into Git 2.38+ and orchestrates multiple Git features for large repo performance:

```
# Clone with Scalar (dramatically faster for large repos)
scalar clone https://github.com/org/monorepo.git
# Internally uses:
# - Partial clone (--filter=blob:none)
# - Sparse checkout
# - Commit graph pre-computation
# - Background maintenance (auto-gc, fetch, commit-graph)
# - FSMonitor (filesystem change monitoring)

# Register an existing repo with Scalar
scalar register

# Run background maintenance
scalar run all

# What Scalar configures automatically:
git config core.fsmonitor true         # filesystem change monitoring
git config core.untrackedcache true    # cache untracked file state
git config feature.manyFiles true      # optimizations for large file
counts
git config fetch.writeCommitGraph true # update commit graph on fetch
```

FSMonitor — Filesystem Change Monitoring

```
# Without FSMonitor: git status rescans all files
# With FSMonitor: OS notifies Git which files changed → status is instant

# Enable FSMonitor (built-in daemon since Git 2.37)
git config core.fsmonitor true
```

```
# Check FSMonitor status
git fsmonitor--daemon status

# Performance impact on large repos:
# git status without FSMonitor: ~5 seconds (large repo)
# git status with FSMonitor:    ~50ms (30-100x faster)
```

Monorepo Tooling Integration with Git

How Nx, Bazel, and Turborepo Use Git

These tools rely on Git for **change detection** — only rebuilding/retesting what changed.

```
# Nx: uses git diff to determine affected projects
nx affected --target=build --base=origin/main --head=HEAD

# Internally runs something like:
CHANGED_FILES=$(git diff --name-only origin/main...HEAD)
# Then maps files to projects using the dependency graph
# Only builds projects that own changed files or depend on them

# Turborepo: similar approach with git hashing for cache keys
turbo run build
# Hashes: git SHA of inputs → cache key
# If cache key matches → skip build (use cached output)

# Bazel: can use git archive or git ls-files for hermetic builds
git ls-files --others --cached | bazel build //...
```

Sparse Checkout + Monorepo CI

```
# GitHub Actions matrix build for monorepo services
strategy:
  matrix:
    service: [payments, auth, notifications, analytics]

steps:
  - uses: actions/checkout@v3
    with:
      filter: blob:none
      sparse-checkout: |
        services/${{ matrix.service }}
        shared/

  - name: Build service
    run: cd services/${{ matrix.service }} && npm run build
```

A4. Git Security

Signing Commits with GPG and SSH Keys

Why Sign Commits

Git allows anyone to set `user.name` and `user.email` to anything — including impersonating your colleagues. Commit signing proves that a commit was made by the holder of a specific private key.

```
Without signing: any git user can set user.email = ceo@company.com
                 git log shows that CEO made a commit they never touched

With signing: the GPG/SSH signature proves possession of a specific key
              GitHub shows "Verified" badge on the commit
```

GPG Commit Signing

```
# Generate a GPG key
gpg --full-generate-key
# Choose: RSA and RSA, 4096 bits, no expiry (or set one)

# List GPG keys
gpg --list-secret-keys --keyid-format=long
# sec   rsa4096/3AA5C34371567BD2 2024-01-01

# Configure Git to use this key
git config --global user.signingkey 3AA5C34371567BD2
git config --global commit.gpgsign true   # auto-sign all commits
git config --global tag.gpgsign true      # auto-sign all tags

# Sign a specific commit (without global setting)
git commit -S -m "feat: add OAuth login"

# Verify a commit's signature
git verify-commit HEAD
git log --show-signature -1 HEAD
```

SSH Key Signing (Git 2.34+, Simpler)

Most developers already have SSH keys. Git 2.34+ allows using them for signing instead of GPG.

```
# Configure SSH signing
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
```

```
# Or use SSH agent key
git config --global user.signingkey "key::ssh-ed25519 AAAAC3..."

# Enable auto-signing
git config --global commit.gpgsign true

# Configure allowed signers file (for verification)
echo "jane@company.com ssh-ed25519 AAAAC3..." >> ~/.ssh/allowed_signers
git config --global gpg.ssh.allowedSignersFile ~/.ssh/allowed_signers

# Verify
git verify-commit HEAD
# Good "git" signature for jane@company.com with ED25519 key SHA256:...
```

Signing Tags and Enforcing Signed Commits

```
# Sign a tag
git tag -s v2.1.0 -m "Release 2.1.0"

# Verify a tag
git verify-tag v2.1.0

# GitHub: require signed commits in branch protection
# Settings → Branches → Branch protection rules → "Require signed commits"
# → unsigned commits are rejected on push

# GitLab: enforce via push rules
# Settings → Repository → Push rules → "Reject unsigned commits"
```

`git verify-commit` and `git verify-tag`

```
# Verify a specific commit
git verify-commit abc1234
# gpg: Signature made Mon 01 Jan 2024 10:00:00 UTC
# gpg:                using RSA key 3AA5C34371567BD2
# gpg: Good signature from "Jane Smith <jane@company.com>"

# Verify a range of commits (useful in CI)
git log --show-signature v2.0.0..HEAD | grep "Good signature\|BAD
signature"

# CI enforcement example
#!/bin/bash
COMMITS=$(git rev-list origin/main..HEAD)
for SHA in $COMMITS; do
  if ! git verify-commit "$SHA" 2>/dev/null; then
    echo "❌ Unsigned commit: $SHA"
    exit 1
```

```
  fi
done
echo "✅ All commits signed"
```

Detecting Secrets Committed to History

Prevention Tools

detect-secrets (Yelp)

```
# Install
pip install detect-secrets

# Scan working directory
detect-secrets scan > .secrets.baseline
detect-secrets audit .secrets.baseline

# As a pre-commit hook
# .pre-commit-config.yaml:
- repo: https://github.com/Yelp/detect-secrets
  hooks:
    - id: detect-secrets
      args: ['--baseline', '.secrets.baseline']
```

gitleaks — Scan Full History

```
# Install
brew install gitleaks

# Scan entire repo history
gitleaks detect --source . --verbose

# Scan only staged changes (pre-commit)
gitleaks protect --staged

# Scan a specific commit range
gitleaks detect --log-opts "main..HEAD"

# As GitHub Actions
- name: Scan for secrets
  uses: gitleaks/gitleaks-action@v2
  env:
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

truffleHog — Deep History Scanner

```
# Install
pip install trufflehog

# Scan full repo history with entropy detection
trufflehog git file://. --only-verified

# Scan a GitHub repo
trufflehog github --repo https://github.com/org/repo

# CI integration
trufflehog git file://. --since-commit HEAD~1 --fail
```

Removing Secrets from History — `git filter-repo` vs BFG

The Core Problem

```
A secret (API key, password) was committed in commit abc1234.
Even after deleting the file and pushing, the secret is:
  - In git history forever (until purged)
  - On every clone ever made
  - On GitHub/GitLab until you contact support

IMMEDIATE ACTIONS:
  1. Revoke the secret (rotate the key/password) ← do this FIRST
  2. Remove from history ← then do this
  3. Force all team members to re-clone ← critical
```

`git filter-repo` (Recommended)

```
# Install
pip install git-filter-repo

# Remove a specific file from ALL history
git filter-repo --path secrets.env --invert-paths

# Remove a specific string (the actual secret) from ALL files in ALL
history
git filter-repo --replace-text replacements.txt
# replacements.txt content:
# AKIAIOSFODNN7EXAMPLE==>REMOVED_API_KEY

# Remove a directory from all history
git filter-repo --path config/secrets/ --invert-paths

# Remove files matching a pattern
git filter-repo --path-regex '.*\.pem$' --invert-paths
```

```
# After filter-repo:
# 1. All object SHAs have changed
# 2. Must force push ALL branches
git push --force --all
git push --force --tags

# 3. Contact GitHub/GitLab to purge their caches
# 4. All collaborators must re-clone (old clones have the secret)
git clone --no-local . ../clean-repo   # fresh clone for yourself
```

BFG Repo Cleaner (Java-based, Older but Faster for Simple Cases)

```
# Install: download BFG jar from rtyley.github.io/bfg-repo-cleaner

# Remove files named secrets.env
java -jar bfg.jar --delete-files secrets.env repo.git

# Replace all occurrences of a string
echo "AKIAIOSFODNN7EXAMPLE" > passwords.txt
java -jar bfg.jar --replace-text passwords.txt repo.git

# Remove files larger than 50MB
java -jar bfg.jar --strip-blobs-bigger-than 50M repo.git

# After BFG, clean up:
cd repo.git
git reflog expire --expire=now --all
git gc --prune=now --aggressive
git push --force --all
```

`git filter-repo` vs BFG Comparison

Feature	`git filter-repo`	BFG
Maintained	✅ Active	⚠ Maintenance mode
Speed	Fast	Faster for simple ops
Complex transformations	✅ Full Python API	Limited
Requirements	Python 3	Java
Recommended by Git project	✅ Yes	No
Handles large repos	✅	✅

`git blame`, CODEOWNERS, and Accountability at Scale

`git blame`

```
# Show who last modified each line of a file
git blame src/auth.js

# Output:
# a3f9d2c (Jane Smith 2024-01-15 10:30:00 +0530  1) function
authenticate(user) {
# b7e1234 (Bob Jones  2024-01-20 14:22:00 +0530  2)   const timeout =
10000;
# a3f9d2c (Jane Smith 2024-01-15 10:30:00 +0530  3)   return fetch(url);

# Blame with line range
git blame -L 10,25 src/auth.js

# Ignore whitespace changes
git blame -w src/auth.js

# Detect moved/copied lines (within file)
git blame -M src/auth.js

# Detect lines moved from other files in this commit
git blame -C src/auth.js

# Blame at a specific commit
git blame abc1234 -- src/auth.js

# Porcelain format (machine-readable, used by IDEs)
git blame --porcelain src/auth.js
```

CODEOWNERS

CODEOWNERS automates PR review requirements — certain files/directories always require review from specific people or teams.

```
# .github/CODEOWNERS (GitHub) or .gitlab/CODEOWNERS (GitLab)

# Global owner
*                           @org/platform-team

# Specific directory
/src/payments/              @org/payments-team @alice

# File type
*.tf                        @org/infra-team

# Specific file
/SECURITY.md                @org/security-team

# Override global rule for specific path
/src/payments/tests/        @org/qa-team   # overrides payments-team for
tests
```

```
# Multiple owners (any one of them can approve)
/src/auth/                  @alice @bob @charlie
```

```
# GitHub enforcement in branch protection:
# Settings → Branches → Require review from Code Owners → ✅

# GitLab enforcement:
# Settings → Repository → Protected Branches → "Require approval from code
owners"
```

Short Interview Answer

> "Commit signing with GPG or SSH keys proves identity — without it, anyone can impersonate any contributor since user.email is just a config string. I enforce signed commits via branch protection. For secrets prevention, I use gitleaks in pre-commit hooks and CI scans. If a secret is committed, `git filter-repo` is the modern tool to purge it from all history — but the secret must be rotated immediately, and all collaborators must re-clone after the force push. CODEOWNERS + required reviews in branch protection ensures the right people review changes to critical code paths."

A5. Git in CI/CD Pipelines ⚠

Shallow Clones and Their Gotchas — Complete CI Guide

(See A3 for full shallow clone details — summary of CI-specific patterns below)

```yaml
# Pattern 1: Pure build/test (no history needed)
- uses: actions/checkout@v3
  with:
    fetch-depth: 1

# Pattern 2: Need git describe for versioning
- uses: actions/checkout@v3
  with:
    fetch-depth: 0   # full history + all tags

# Pattern 3: Monorepo change detection
- uses: actions/checkout@v3
  with:
    filter: blob:none   # blobless: full history without binary content
    fetch-depth: 0

# Pattern 4: Security scans (need full history for secret detection)
- uses: actions/checkout@v3
```

```
  with:
    fetch-depth: 0   # truffleHog/gitleaks need full history
```

Why Tags Don't Fetch by Default in CI

The Problem

```
# Standard CI clone:
git clone --depth=1 https://repo.git

# Result: NO tags fetched. git describe fails.
git describe
# fatal: No names found, cannot describe anything.

# Why: --depth=1 downloads 1 commit, no history, no tags reachable from it
# Even git fetch doesn't get tags by default in some configurations
```

The Fix

```
# Fix 1: Fetch tags explicitly
git fetch --tags --unshallow

# Fix 2: During clone, fetch all
git clone --no-shallow-tags https://repo.git   # not standard
# Better:
git clone https://repo.git && git fetch --tags

# Fix 3: In GitHub Actions
- uses: actions/checkout@v3
  with:
    fetch-depth: 0       # full clone, all tags included

# Fix 4: Fetch only what's needed
git fetch --depth=1 origin '+refs/tags/*:refs/tags/*'
# Fetches all tag refs without fetching the history they point to
```

Semantic-Release and Tags in CI

```
# semantic-release needs full tag history to determine version bumps
# Without tags: fails to compute next version
# GitHub Actions fix:
- uses: actions/checkout@v3
  with:
    fetch-depth: 0
    persist-credentials: false  # for token-based tag pushing
```

```
- name: Release
  run: npx semantic-release
  env:
    GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

Detached HEAD in CI Pipelines — Causes and Handling

Why CI Pipelines Are Almost Always in Detached HEAD

```
# GitHub Actions checkout:
actions/checkout → git checkout --detach abc1234
# HEAD = commit SHA, not branch name

# Why: CI checks out a specific commit SHA (the one that triggered the
build)
# The branch may have moved since the trigger

cat .git/HEAD
# abc1234e7b18f6d...  ← detached HEAD (SHA, not branch ref)
```

Problems Caused by Detached HEAD in CI

```
# Problem 1: git push fails
git push origin HEAD
# fatal: HEAD does not point to a branch

# Problem 2: git branch shows no current branch
git branch
# * (HEAD detached at abc1234)

# Problem 3: Scripts that use git symbolic-ref HEAD fail
git symbolic-ref HEAD
# fatal: HEAD is not a symbolic ref

# Problem 4: git describe may not find tags from detached position
git describe
# may work but context-dependent
```

Handling Detached HEAD in CI Scripts

```
#!/bin/bash
# Robustly get the current branch name in CI

# Method 1: From environment variables (most reliable in CI)
```

```
BRANCH=${GITHUB_REF_NAME:-}           # GitHub Actions
BRANCH=${CI_COMMIT_REF_NAME:-}        # GitLab CI
BRANCH=${BRANCH_NAME:-}               # Jenkins (if configured)

# Method 2: From git (works when not detached)
BRANCH=$(git symbolic-ref --short HEAD 2>/dev/null)

# Method 3: Fallback — find branches containing current commit
if [ -z "$BRANCH" ]; then
  BRANCH=$(git branch -r --contains HEAD | grep -v HEAD | head -1 | tr -d
' ' | sed 's/origin\///')
fi

echo "Building branch: $BRANCH"

# Method 4: Re-attach HEAD (when you need to push from CI)
git checkout -b ci-branch-$BUILD_NUMBER
git push origin ci-branch-$BUILD_NUMBER
```

`git bisect` — Manual and Automated Regression Hunting

What it is

`git bisect` performs a **binary search** through commit history to find the exact commit that introduced a bug or regression. Given N commits, it needs only $\log_2(N)$ steps.

```
1000 commits to search → 10 bisect steps to find the culprit
1,000,000 commits      → 20 bisect steps
```

Manual Bisect

```
# Scenario: Production is broken. You know it worked at v2.0.0.

# Start bisect
git bisect start

# Mark current HEAD as bad (broken)
git bisect bad

# Mark a known good commit (where it worked)
git bisect good v2.0.0

# Git checks out the middle commit automatically:
# Bisecting: 142 revisions left to test after this (roughly 7 steps)
# [abc1234] feat: add payment gateway

# Test manually, then tell Git the result:
# If this commit is broken:
```

```
git bisect bad
# If this commit works:
git bisect good

# Repeat until Git identifies the culprit:
# abc1234 is the first bad commit
# commit abc1234
# Author: Jane Smith <jane@co.com>
# Date: ...
# feat: add payment gateway

# Exit bisect mode (restores HEAD to original position)
git bisect reset
```

Automated Bisect

```
# Provide a test script that exits 0 for good, non-zero for bad
cat > /tmp/test.sh << 'EOF'
#!/bin/bash
npm test -- --grep "payment processing"
exit $?
EOF
chmod +x /tmp/test.sh

# Automated bisect runs the script on each candidate commit
git bisect start
git bisect bad HEAD
git bisect good v2.0.0
git bisect run /tmp/test.sh

# Git will find the culprit commit automatically
# git bisect reset is called automatically when done
```

Bisect with Build Step

```
#!/bin/bash
# test.sh — used with git bisect run

# Build (exit 125 = skip this commit, not relevant)
if ! npm run build 2>/dev/null; then
  exit 125   # 125 = "skip" — this commit can't be tested (build broken)
fi

# Test
npm test -- --grep "checkout"
# exit 0 = good, 1 = bad, 125 = skip
```

Using Bisect for Performance Regressions

```bash
#!/bin/bash
# perf-test.sh — find when response time regressed

RESPONSE_TIME=$(curl -o /dev/null -s -w "%{time_total}"
http://localhost:3000/api/checkout)
THRESHOLD=0.200   # 200ms

if (( $(echo "$RESPONSE_TIME > $THRESHOLD" | bc -l) )); then
  echo "SLOW: ${RESPONSE_TIME}s"
  exit 1
fi
echo "FAST: ${RESPONSE_TIME}s"
exit 0
```

Merge Queues — GitHub Merge Queue, GitLab Merge Trains

The Problem Merge Queues Solve

```
Scenario: 20 PRs all approved and ready to merge.
All were tested against the same base (main at SHA abc1234).
You merge them one by one.

Problem: PR #5 might conflict with PR #3 which just merged.
         "Tested" doesn't mean "tested together."
         Each new merge may break main.

This is called "the late integration problem."
```

GitHub Merge Queue

```
How it works:
1. Developer marks PR "ready to merge"
2. PR enters the merge queue (not merged yet)
3. GitHub creates a virtual merge of: main + all queued PRs ahead of this
one
4. CI runs on this virtual merge
5. If CI passes → PR is merged to main
6. If CI fails → failing PR is removed from queue, others continue
```

```
# Enable in GitHub repository settings:
# Settings → Branches → Branch protection rules → main
# ✅ Require merge queue

# Optional: GitHub Actions triggered by merge_group event
```

```yaml
on:
  merge_group:
    types: [checks_requested]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm test
```

GitLab Merge Trains

```
GitLab's equivalent to Merge Queue.
Merge trains run CI on batches of PRs merged together.
If the batch passes, all are merged.
If the batch fails, GitLab bisects to find the failing MR.

More aggressive batching → fewer CI runs → faster throughput
```

```yaml
# Enable in GitLab:
# Settings → Merge Requests → Merge method → Merge trains

# .gitlab-ci.yml — runs in merge train context
merge-train-check:
  only:
    - merge_trains
  script:
    - npm test
    - npm run integration-test
```

Merge Queue vs Traditional Merge Strategies

Approach	Queue	Testing	Safety
Direct merge	None	Against old base	Low
Required CI (standard)	None	Against base at PR open time	Medium
Up-to-date branch requirement	None	Against latest main	Better
Merge Queue	FIFO queue	Against virtually merged state	High
Merge Train	Batch queue	Against batched merge state	Highest throughput

Branch Naming Conventions Driving Pipeline Behavior

Conventions That Trigger Specific Pipeline Behaviors

```
# Jenkins declarative pipeline — branch-based routing
pipeline {
  stages {
    stage('Test') {
      steps { sh 'npm test' }
    }
    stage('Deploy to Staging') {
      when {
        branch 'release/*'
      }
      steps { sh './deploy.sh staging' }
    }
    stage('Deploy to Production') {
      when {
        branch 'main'
      }
      steps { sh './deploy.sh production' }
    }
    stage('Publish Hotfix') {
      when {
        branch 'hotfix/*'
      }
      steps {
        sh './deploy.sh production'
        sh './notify-oncall.sh'
      }
    }
  }
}
```

```
# GitLab CI — rules-based branch routing
deploy-production:
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
  script:
    - ./deploy.sh production

deploy-staging:
  rules:
    - if: '$CI_COMMIT_BRANCH =~ /^release\//'
  script:
    - ./deploy.sh staging

feature-test:
  rules:
    - if: '$CI_COMMIT_BRANCH =~ /^(feature|fix|chore)\//'
  script:
    - npm test
    - npm run lint
```

Common Naming Conventions by Tool

```
GitFlow:
  main, master      → production deploy
  develop           → staging deploy
  feature/*         → CI test only
  release/*         → staging deploy + release prep
  hotfix/*          → immediate production deploy

GitHub Flow:
  main              → production deploy
  *                 → CI test + deploy to review environment

Conventional:
  feat/*            → test + deploy to preview
  fix/*             → test + deploy to preview
  chore/*           → test only (no deploy)
  docs/*            → skip CI (or docs-only build)
```

Short Interview Answer

> "In CI/CD, the three most important Git gotchas are: shallow clones breaking `git describe` and tag-based versioning (fix with `fetch-depth: 0`), detached HEAD state making branch-aware scripts fail (use CI environment variables for branch names), and secrets accidentally committed to history (detect with gitleaks, remove with `git filter-repo`). For merge safety at scale, merge queues solve the late integration problem by running CI on the virtual merged state before any PR actually touches main."

A6. Git Workflows at Scale

How Google's Internal Monorepo (Piper) Compares to Git at Scale

Google's Piper

Google uses **Piper** — a custom centralized VCS built on top of Bigtable — not Git, for its main monorepo.

```
Scale:
  - 2 billion lines of code
  - 86 terabytes of content
  - 25,000 engineers committing daily
  - 45,000 commits/day
  - 250 million files

Key differences from Git:
  - Centralized (like SVN, not distributed)
  - Enforces linear history (no branching at scale in the main repo)
```

```
  - "Workspaces" instead of branches
  - Cloud-based storage (files streamed from servers, not full local copy)
  - Built-in code review (Critique) tightly integrated
  - "CitC" (Clients in the Cloud) — working copy is virtual, stored
server-side
```

Why Google Doesn't Use Git for Piper

```
1. Distribution overhead: 86TB × thousands of engineers = impractical
2. Merge complexity: Merging 250M files is infeasible in Git
3. Tooling integration: Piper is tightly coupled to Google's build
(Blaze/Bazel),
   test, code review, and deployment systems
4. Atomicity at scale: Piper supports atomic multi-file, multi-project
commits
   across the entire codebase
```

Microsoft's Approach — Git at Windows Scale

Microsoft uses Git for Windows OS development (the repo that drove VFS for Git and Scalar). Key stats: 3.5M files, 300GB, 4,000 engineers.

Solutions developed: GVFS (virtual filesystem), Scalar, commit graph, FSMonitor — most now upstreamed into Git itself.

Stacked PRs and Stacked Diffs — Graphite, ghstack

The Problem with Large Features

Large features that touch many files are hard to review as single PRs. But breaking them into independent PRs means waiting for each to merge before starting the next.

Stacked PRs solve this: each PR builds on the previous one, reviewable independently, mergeable in order.

```
main ← PR1 (foundation) ← PR2 (service layer) ← PR3 (API endpoints) ← PR4
(UI)

Benefits:
- Each PR is small and reviewable
- PR4 can be written while PR1 is still in review
- Merging is ordered: PR1 → PR2 → PR3 → PR4
```

`ghstack` (Facebook's Tool)

```
```

```
# Install
pip install ghstack

# Push a stack of commits as separate PRs automatically
ghstack push

# Each local commit becomes a separate PR on GitHub
# PRs are linked and ordered
# Updating: modify commits, re-run ghstack push
# → All PRs in the stack are updated simultaneously
```

Graphite (Commercial Tool for Stacked PRs)

```
# Install
brew install withgraphite/tap/gt

# Create stacked branches
gt checkout -b pr1/foundation
# work, commit
gt checkout -b pr2/service-layer   # branches from pr1/foundation
# work, commit
gt checkout -b pr3/api-endpoints   # branches from pr2
# work, commit

# Submit the stack as PRs
gt submit --stack

# Update all PRs after rebase
gt sync   # fetches main, rebases entire stack
gt submit # updates all PRs

# Visualize the stack
gt log
# ◉ pr3/api-endpoints
# ◉ pr2/service-layer
# ◉ pr1/foundation
# ○ main
```

Managing Stacked PRs Manually (Without Tools)

```
# Create stacked branches
git checkout -b feat/part1 main
git commit -m "feat: part 1 foundation"

git checkout -b feat/part2 feat/part1
git commit -m "feat: part 2 service layer"

git checkout -b feat/part3 feat/part2
```

```
git commit -m "feat: part 3 API"

# Push all and open PRs:
# PR1: feat/part1 → main
# PR2: feat/part2 → feat/part1
# PR3: feat/part3 → feat/part2

# When PR1 merges:
git fetch origin
git checkout feat/part2
git rebase origin/main   # now based on actual main
git push --force-with-lease

git checkout feat/part3
git rebase feat/part2
git push --force-with-lease
```

CODEOWNERS Automation, Required Reviewers, Policy Enforcement

Advanced CODEOWNERS Patterns

```
# .github/CODEOWNERS

# Global fallback owner
*                           @org/senior-engineers

# Security-critical files always require security team review
/src/auth/                  @org/security-team
/src/crypto/                @org/security-team
*.pem                       @org/security-team
*.key                       @org/security-team

# Infrastructure changes require SRE review
*.tf                        @org/sre-team
*.yml                       @org/sre-team
Dockerfile                  @org/sre-team
/k8s/                       @org/sre-team

# API contract files require API review team
/api/openapi.yaml           @org/api-team @org/backend-leads

# Shared libraries require library owners
/libs/                      @alice @bob   # named individuals for critical
shared code

# Documentation only — any contributor can approve
/docs/                      @org/tech-writers

# Test files — QA team review
/tests/e2e/                 @org/qa-team
```

```
# Override: payments tests can be reviewed by either team
/tests/e2e/payments/        @org/qa-team @org/payments-team
```

Programmatic Policy Enforcement

```
# GitHub Actions: enforce custom review policies beyond CODEOWNERS
name: Policy Enforcement

on:
  pull_request:
    types: [opened, synchronize]

jobs:
  check-reviewers:
    runs-on: ubuntu-latest
    steps:
      - name: Check breaking change requires architect sign-off
        uses: actions/github-script@v6
        with:
          script: |
            const files = await github.rest.pulls.listFiles({
              owner: context.repo.owner,
              repo: context.repo.repo,
              pull_number: context.issue.number
            });
            const hasAPIChange = files.data.some(f =>
f.filename.includes('openapi'));
            if (hasAPIChange) {
              // Check if architect has approved
              const reviews = await github.rest.pulls.listReviews({...});
              const architectApproved = reviews.data.some(r =>
                r.user.login === 'chief-architect' && r.state ===
'APPROVED'
              );
              if (!architectApproved) {
                core.setFailed('API changes require architect approval');
              }
            }
```

Protected Branches, Required Status Checks, and Enforcement Strategies

Complete Branch Protection Configuration (GitHub)

```
# GitHub branch protection for 'main' — enterprise configuration:

Protect matching branches: main
```

```
✅ Require a pull request before merging
  ✅ Require approvals: 2 (for main)
  ✅ Dismiss stale pull request approvals when new commits are pushed
  ✅ Require review from Code Owners
  ✅ Require approval of the most recent reviewable push
  ❌ Allow specified actors to bypass required pull requests  # no bypass

✅ Require status checks to pass before merging
  ✅ Require branches to be up to date before merging
  Required status checks:
    - ci/test (GitHub Actions)
    - ci/lint
    - ci/security-scan
    - codecov/patch (coverage check)
    - sonarcloud (code quality)

✅ Require conversation resolution before merging
✅ Require signed commits
✅ Require linear history  # prevents merge commits (forces squash or
rebase merge)
✅ Require deployments to succeed before merging
  - staging environment must be healthy

✅ Restrict who can push to matching branches
  - Allow push: @org/release-managers (for emergency hotfixes only)

❌ Allow force pushes   # DISABLED
❌ Allow deletions       # DISABLED

✅ Require merge queue
  Merge method: Squash and merge
  Merge strategy: All green (require all checks to pass)
  Maximum pull requests to build: 5
```

Rulesets (GitHub Enterprise) — More Powerful

```
# GitHub Rulesets (newer, more powerful than branch protection)
# Can apply to multiple branches at once, with bypass lists

ruleset:
  name: "Production safeguards"
  target: "branch"
  enforcement: "active"   # or "evaluate" (audit mode) or "disabled"

  conditions:
    ref_name:
      include: ["refs/heads/main", "refs/heads/release/*"]
      exclude: []

  rules:
    - type: "deletion"      # prevent deletion
```

```
      - type: "non_fast_forward"  # prevent force push
      - type: "required_linear_history"
      - type: "required_signatures"
      - type: "pull_request"
        parameters:
          required_approving_review_count: 2
          dismiss_stale_reviews_on_push: true
          require_code_owner_review: true
      - type: "required_status_checks"
        parameters:
          strict_required_status_checks_policy: true
          required_status_checks:
            - context: "ci / test"
            - context: "security / scan"

    bypass_actors:
      - actor_type: "Team"
        actor_id: 1234    # SRE team can bypass in emergencies
        bypass_mode: "always"
```

Managing Hundreds of Repos — Fleet-Level Git Operations

The Challenge

Organizations with 100–1000 repos face operational overhead:

- Applying a security patch to a dependency across all repos
- Updating CI configuration templates uniformly
- Enforcing consistent branch protection rules
- Finding all repos with a specific vulnerability

Tools for Fleet Management

`gh` CLI + scripting

```
# List all repos in an org
gh repo list myorg --limit 1000 --json name --jq '.[].name'

# Clone all repos
gh repo list myorg --limit 1000 --json sshUrl --jq '.[].sshUrl' | xargs -P
10 -n 1 git clone

# Apply a fix to all repos that match a pattern
gh repo list myorg --topic "node-service" --json name --jq '.[].name' |
while read repo; do
  cd "$repo"
  # Apply patch, commit, push, open PR
  sed -i 's/node:18-alpine/node:20-alpine/g' Dockerfile
  git commit -am "chore: upgrade Node.js to 20"
  gh pr create --title "chore: upgrade Node.js 20" --body "Security
```

```
update"
  cd ..
done
```

Multiclone / all-repos tools

```
# all-repos: Python tool for managing multiple repos
pip install all-repos
# config: all-repos.json lists all repos to manage

all-repos-clone       # clone all repos
all-repos-grep "TODO" # grep across all repos
all-repos-sed -i 's/old/new/g' -- '*.js'  # sed across all repos
all-repos-push        # commit and push changes across all repos
```

Terraform for GitHub/GitLab settings

```
# Manage branch protection at scale with Terraform
resource "github_branch_protection" "main" {
  for_each       = toset(var.all_repos)
  repository_id  = each.key
  pattern        = "main"

  required_pull_request_reviews {
    required_approving_review_count = 2
    require_code_owner_reviews      = true
    dismiss_stale_reviews           = true
  }

  required_status_checks {
    strict   = true
    contexts = ["ci / test", "security / scan"]
  }

  allows_force_pushes = false
  allows_deletions    = false
}
```

A7. Troubleshooting and Recovery ⚠

Recovering Lost Commits with `git reflog` — Step by Step

(See A1 for reflog internals — detailed recovery scenarios below)

The Master Recovery Flowchart

```
"I lost my work"
        |
        ▼
Did you commit it?
  NO → git stash list / check git status / git diff
  YES ↓

Is it in the reflog?
  git reflog ← search here first
        |
  FOUND ↓                    NOT FOUND ↓
        |                          |
git reset --hard SHA        git fsck --unreachable
OR git checkout -b          → search orphaned objects
   recover SHA                     |
                             git show <sha>
                             git cherry-pick <sha>
```

Full Recovery Reference

```
# Universal first step: look at reflog
git reflog --all | head -50

# Find commits that match what you remember
git log --all --oneline | grep "payment"

# Find a commit by its content
git log --all -S "function authenticate" --oneline

# Find a commit by message
git log --all --grep="OAuth" --oneline

# Find ALL unreachable objects (after gc hasn't run)
git fsck --unreachable | grep "unreachable commit"
# unreachable commit abc1234...

# Inspect an unreachable commit
git show abc1234

# Recover it
git checkout -b rescued-branch abc1234
# or:
git cherry-pick abc1234
```

Recovering a Deleted Branch

```
# Scenario: git branch -D feature/payments (deleted unmerged branch)

# Method 1: Reflog
git reflog | grep "feature/payments"
# HEAD@{5}: checkout: moving from feature/payments to main
# OR check the branch's own reflog:
git reflog show feature/payments
# feature/payments@{0}: commit: feat: final payment implementation

# Recreate the branch
git checkout -b feature/payments HEAD@{5}
# or with the exact SHA:
git checkout -b feature/payments abc1234

# Method 2: Search git log --all
git log --all --oneline | head -20
# (deleted branch commits may still be here before gc runs)

# Method 3: git fsck if reflog doesn't help
git fsck --unreachable | grep commit | awk '{print $3}' | while read sha;
do
  git log --oneline -1 $sha
done
# Look for commits matching the deleted branch's work
```

Undoing a Bad Merge to Main Without Losing History

```
# Scenario: merged a feature branch into main, it broke production

# Option 1: Revert the merge commit (PREFERRED — preserves history)
git log --oneline -5
# abc1234 Merge feature/bad-feature into main  ← the bad merge
# def5678 feat: add logging

git revert -m 1 abc1234
# -m 1 = revert to "first parent" (main before the merge)
# Creates a new commit that undoes the merge

git push origin main   # safe, no history rewrite

# ⚠ IMPORTANT: If you later want to re-merge the feature branch after
fixing it,
# you MUST revert the revert first:
git revert HEAD  # reverts the revert commit
# THEN fix the feature branch
# THEN merge again

# Option 2: Reset (if no one else has pulled)
git reset --hard HEAD~1    # moves main back one commit
```

```
git push --force-with-lease origin main
# ⚠ DANGEROUS: anyone who pulled main now has a diverged history

# When to use which:
# Revert → shared branch, other engineers have pulled, audit trail
required
# Reset  → you are 100% certain no one else has fetched the merge commit
```

Recovering from Accidental `git reset --hard`

```
# Scenario: git reset --hard HEAD~10 (lost 10 commits)

# Step 1: Don't panic, don't run any other destructive commands
# The commits are still in the object store (until gc runs)

# Step 2: Find them in the reflog
git reflog
# HEAD@{0}: reset: moving to HEAD~10
# HEAD@{1}: commit: feat: add analytics dashboard  ← this is what you want
# ...

# Step 3: Reset back to before the accident
git reset --hard HEAD@{1}
# Working directory and staging area restored to the pre-reset state

# Verify
git log --oneline -5
# Should show the 10 recovered commits

# If reflog doesn't help (very recent gc):
git fsck --lost-found
# Writes unreachable objects to .git/lost-found/
ls .git/lost-found/commit/
# Files named by SHA — inspect each with git show
```

Diagnosing Repository Corruption with `git fsck`

What `git fsck` Checks

```
# Full repository check
git fsck --full

# What it verifies:
# - Object connectivity: every referenced object exists
# - Object integrity: every object's SHA matches its content
# - Dangling/unreachable objects: commits/blobs not reachable from any ref
# - Missing objects: refs pointing to non-existent SHAs
```

```
# Output interpretation:
git fsck --full
# Checking object connectivity and validity.
# checking connectivity: done.
# dangling commit abc1234  ← unreachable (normal after rebase/reset)
# dangling blob def5678    ← unreachable blob (from deleted tracked file)
# missing tree 9f8e7d6c    ← THIS IS A PROBLEM — a tree object is gone
# broken link from commit abc1234 to tree xyz  ← CORRUPTION

# Strict check (more pedantic)
git fsck --strict

# Unreachable objects (dangling = normal, not errors)
git fsck --unreachable

# Only errors (suppress dangling warnings)
git fsck 2>&1 | grep -v "dangling"
```

Common Corruption Scenarios and Fixes

```
# Scenario 1: "object file is empty" (disk write error, power loss)
# .git/objects/ab/cdef1234 exists but is 0 bytes

# Fix: restore from remote
git fetch origin   # re-downloads the missing/corrupt objects

# Scenario 2: "loose object is corrupt" (filesystem corruption)
# SHA of stored object doesn't match computed SHA

# Fix: remove corrupt object, fetch from remote
rm .git/objects/ab/cdef1234
git fetch origin

# Scenario 3: Packfile corruption
git verify-pack -v .git/objects/pack/pack-abc.pack
# If this fails, the packfile is corrupt

# Fix: re-clone from remote (safest)
cd ..
git clone git@github.com:org/repo.git repo-clean

# Scenario 4: Missing HEAD reference
cat .git/HEAD
# (empty or missing)

# Fix: recreate HEAD
echo "ref: refs/heads/main" > .git/HEAD

# Scenario 5: index corruption (git add/status broken)
# "fatal: index file corrupt"
```

```
rm .git/index
git reset HEAD   # recreates index from last commit
```

Prevention

```
# Enable object fsck on fetch and push
git config --global fetch.fsckObjects true
git config --global receive.fsckObjects true
git config --global transfer.fsckObjects true

# Periodic health check in CI
git fsck --no-dangling
if [ $? -ne 0 ]; then
  echo "Repository corruption detected!"
  exit 1
fi
```

`git bisect` for Finding the Exact Commit That Broke Production

(Detailed in A5 — production-focused patterns below)

```
# Production scenario: deploy went out, monitoring alerts firing
# Last known good: the tag deployed yesterday

# Fast start: last known good is yesterday's tag
git bisect start HEAD v2.1.0   # bad HEAD, good v2.1.0

# Automated test against production health check
cat > /tmp/health-check.sh << 'EOF'
#!/bin/bash
npm run build > /dev/null 2>&1 || exit 125   # skip if build fails
npm run start &
SERVER_PID=$!
sleep 3
STATUS=$(curl -s -o /dev/null -w "%{http_code}"
http://localhost:3000/health)
kill $SERVER_PID

if [ "$STATUS" = "200" ]; then exit 0; fi
exit 1
EOF

git bisect run /tmp/health-check.sh

# After finding the culprit:
# abc1234 is the first bad commit
# Author: John Doe
# Date: Yesterday at 4:47pm
```

```
# chore: upgrade dependencies
# → Likely culprit: dependency upgrade broke something

git bisect reset

# Now you can revert just that commit
git revert abc1234
git push origin main
```

Short Interview Answer

> "My recovery toolkit in order: reflog first (`git reflog` shows every position HEAD has been), then `git fsck --unreachable` to find orphaned commits the reflog might have missed, then `git cherry-pick` or `git reset --hard` to the recovered SHA. For production breakages, `git bisect run` with an automated test script finds the culprit commit in O(log N) steps — I've used this to find regressions in repos with 50,000+ commits in under 20 binary search steps. For repository corruption, `git fsck --full` diagnoses the problem, and `git fetch` from the remote usually restores missing objects."

A8. Git System Design (Senior/Staff Level)

> These questions appear in principal engineer, staff SRE, and platform engineering interviews. The answer demonstrates both Git mastery and distributed systems thinking.

How Would You Design Git's Object Storage from Scratch?

The Requirements

```
Requirements:
  1. Store arbitrary content (files, directories, commits, tags)
  2. Detect duplicate content (efficiency)
  3. Verify integrity (no silent corruption)
  4. Support distributed cloning (multiple full copies)
  5. Immutable history (can't secretly change past commits)
  6. Efficient transfer (don't re-send content the client already has)
```

The Design

Content-Addressable Store (CAS)

The core insight: use a hash of the content as its address. This gives you deduplication and integrity verification in one mechanism.

```
Storage interface:
  write(content) → hash
  read(hash) → content
  exists(hash) → bool

Hash function requirements:
  - Deterministic (same content → same hash always)
  - Collision-resistant (different content → different hash, practically
always)
  - Fast to compute
  - Fixed output size regardless of input

Git chose: SHA-1 (40 hex chars)
Migrating to: SHA-256 (64 hex chars)
```

Object Types as an Overlay

```
On top of the raw CAS, define typed objects:
  blob    = CAS(content of a file)
  tree    = CAS(list of {mode, name, hash} entries)
  commit  = CAS(tree-hash + parent-hash(es) + metadata + message)
  tag     = CAS(target-hash + tagger + message + optional signature)

Type prefix in hash input:
  SHA1("blob 1234\0<content>")   ← type + size + null + content
  SHA1("tree 89\0<entries>")

This means the hash space is shared but type-checked — a tree SHA cannot
accidentally collide with a blob SHA for the same content.
```

The DAG Emerges Naturally

```
Commit hashes include parent hashes.
If you change any ancestor commit, all descendant hashes change.
This creates an immutable, tamper-evident chain — exactly like a
blockchain.

Historical tampering is detectable without a central authority:
Two different clones of the same repo with the same HEAD SHA
are GUARANTEED to have identical history.
```

Deduplication

```
Two files with identical content → one blob.
Binary stored once even if committed 1000 times.
Only need to store the pointer (hash) in the tree, not the content again.
```

Packfiles for Efficiency

```
Problem: millions of individual files in .git/objects/ → filesystem
overhead
Solution: pack related objects into one file (packfile)

Packfile design:
  - Topologically sorted (bases before deltas)
  - Delta encoding: similar objects stored as diffs (one direction)
  - Index file: hash → byte offset in pack (O(log n) lookup via binary
search)
  - Max delta depth: avoids long reconstruction chains

Pack negotiation for transfer:
  Client: "I want A, B, C. I have X, Y, Z."
  Server: computes delta chain from known to wanted
  Server: sends minimal packfile
  → Only transfer what the client doesn't have
```

How Does Conflict Detection Work Internally?

Three-Way Merge Algorithm

```
Git never compares two files — it compares THREE:
  1. Base (common ancestor)
  2. Ours (current branch)
  3. Theirs (incoming branch)

For each hunk (contiguous region) in the diff:

  If only OURS changed → take OURS (no conflict)
  If only THEIRS changed → take THEIRS (no conflict)
  If BOTH changed in the same region → CONFLICT
  If BOTH made the same change → take either (no conflict — Git auto-
resolves)
  If unchanged in both → keep as-is
```

```
# See the three-way merge in action:
git checkout main
git merge feature/login

# If conflict:
# <<<<<<< HEAD (our version)
# const timeout = 5000;
# =======
# const timeout = 10000;
```

```
# >>>>>>> feature/login (their version)

# Base was: const timeout = 3000;
# Both changed timeout — conflict

# To see the base version:
git show :1:src/config.js  # 1 = base, 2 = ours, 3 = theirs
git show :2:src/config.js
git show :3:src/config.js
```

The ORT Algorithm (Git's Current Default)

```
ORT = "Ostensibly Recursive's Twin" (default since Git 2.33)

Improvements over 'recursive':
  1. Handles rename detection more accurately
  2. Doesn't modify the working tree during merge (uses index only)
     → Faster, and failures don't leave partial state
  3. Better handling of criss-cross merges
  4. More correct handling of directory renames
  5. Significantly faster on large repos

Criss-cross merge (why 'recursive' was needed over 'resolve'):
  A ← C
  ↑   ↑
  B ← D  (C and D both branch from A/B, then cross-merge)
  Multiple common ancestors → must merge the ancestors first → virtual
merge base
```

Conflict Detection for Renames

```
# Git detects renames by content similarity (not by tracking)
# A file deleted + a similar file added = rename

# Rename detection threshold (default 50% similarity)
git merge -X rename-threshold=70%

# See what renames Git detected
git diff --name-status --diff-filter=R HEAD~1
```

How Does GitHub Scale Git Hosting to Millions of Repos?

The Architecture Problem

```
GitHub hosts 100M+ repos.
At peak: millions of git fetch/push operations per minute.
A single git server can handle ~1,000 concurrent operations.

Challenges:
  1. Read scaling: many concurrent git fetch operations
  2. Write serialization: pushes to the same repo must be ordered
  3. Consistency: after push, all reads must see the new commits
  4. Reliability: repo data must survive hardware failures
  5. Geo-distribution: fast access from anywhere on Earth
```

Key Architectural Decisions

1. Repository Routing

```
Every repo has a "home" server determined by consistent hashing.
All operations for a repo are routed to the same server cluster.
This ensures write serialization (no split-brain writes).

github.com → load balancer → routing layer (consistent hash)
           → repo server cluster for "org/repo"
           → git-receive-pack or git-upload-pack
```

2. Read Replicas

```
Reads (git fetch, git clone) don't need the primary server.
Git objects are immutable after write → safe to serve from replicas.

Architecture:
  Primary: handles writes, propagates to replicas
  Replicas: serve reads, eventual consistency (seconds lag)

After a push, new objects propagate to replicas before the push response
is returned to the client → "read your own writes" consistency guarantee.
```

3. Spokes Architecture (GitHub's Published Design)

```
GitHub uses a "Spokes" system:
  - Every repo has 3+ replicas across different datacenters
  - Writes go to all 3 simultaneously (Paxos-like agreement)
  - Reads can be served from any replica
  - GitHub can survive losing any 2 replicas without data loss
```

4. DGit (Distributed Git)

```
GitHub's internal system:
  - Git objects stored in a distributed object store (similar to S3)
  - Pack files stored as blobs in the object store
  - Metadata (refs, branch tips) stored in a distributed database
  - Serving layer assembles git responses from these stores

Advantages:
  - Horizontal scaling (add more serving nodes)
  - Storage decoupled from serving
  - Efficient caching (popular repos' packs cached in RAM)
```

What Happens Under the Hood During a `git push` End to End?

Complete `git push origin main` Walkthrough

```
git push origin main
```

Step 1: Resolve remote and authenticate

```
1. Read .git/config to find 'origin' URL
2. Parse URL: SSH (git@github.com:org/repo.git) or HTTPS
3. SSH: establish connection, authenticate via key exchange
   HTTPS: establish TLS, authenticate via credential helper
```

Step 2: Reference discovery

```
4. Client connects to remote
5. Server runs: git-receive-pack /org/repo.git
6. Server sends: list of all refs and their current SHAs
   + capability advertisement (side-band, report-status, etc.)

   Example server response:
   a3f9d2c refs/heads/main
   b7e1234 refs/heads/develop
   c4d5678 refs/tags/v2.0.0
```

Step 3: Push negotiation

```
7. Client sends: "I want to update refs/heads/main from a3f9d2c to
d5e6789"
8. Client sends: "I have these objects already: a3f9d2c, b7e1234..."
9. Server computes: which objects does the client have that I don't?
   (walks DAG from d5e6789 backward until hitting objects server has)
```

Step 4: Packfile transfer

```
10. Client creates a packfile containing only the NEW objects
    (objects in d5e6789's ancestry that server doesn't have)
11. Delta compression: find similar objects server already has, send
deltas
12. Client sends packfile to server (with progress reporting via side-
band)
```

Step 5: Server validation

```
13. Server receives packfile
14. Server verifies every object's SHA integrity (if
receive.fsckObjects=true)
15. Server checks branch protection rules (via hooks)
16. Server runs pre-receive hook (reject if returns non-zero)
```

Step 6: Ref update

```
17. Server checks push is a fast-forward (or force push is authorized)
    Fast-forward: new commit is a descendant of current tip → safe
    Non-fast-forward: would lose commits → rejected unless --force

18. Server atomically updates refs/heads/main to d5e6789
19. Server runs update hook (per-ref validation)
20. Server runs post-receive hook (notifications, CI triggers)
```

Step 7: Response to client

```
21. Server sends status: success or error with message
22. Client updates remote-tracking ref:
    refs/remotes/origin/main → d5e6789
23. Connection closed
```

How Does Git Guarantee Data Integrity Across Clones?

```
The guarantee:
  Two repositories with the same HEAD SHA are GUARANTEED to have
  identical content and history — without any central authority.

How:
```

```
1. Content-addressable storage
   Every object's identity IS its content hash.
   Same content → same hash, different content → different hash.
   (With 1-in-10^48 collision probability for SHA-1)

2. Chained hashes
   Commit SHA = hash of (tree SHA + parent SHA + message)
   Tree SHA = hash of (blob SHAs + filenames)
   → Changing any file changes its blob SHA
     → changes the tree SHA referencing it
       → changes the commit SHA referencing the tree
         → changes all descendant commit SHAs

3. Distributed verification
   Every clone has the full object graph.
   git fsck can verify integrity without consulting any server.
   No trust required — math proves consistency.

4. Transfer integrity
   Every received object is verified:
   received_content's SHA must equal the reference SHA
   If any bit flips in transit → SHA mismatch → rejected
   No need for a checksum layer on top (SHA IS the checksum)

5. Signature layer (optional)
   GPG/SSH signed commits add identity guarantees on top:
   Not just "this content is correct" but "this specific person approved
it"
```

Designing a Branching Strategy for 200 Engineers on One Repo

The Constraints

```
200 engineers:
  - Multiple teams (payments, auth, recommendations, infrastructure...)
  - Multiple release cadences (some teams ship daily, some monthly)
  - Main must be deployable at all times
  - CI/CD must remain fast (< 10 min feedback loop)
  - Code review must be required but not a bottleneck
  - Rollback must be instant
```

The Recommended Architecture

Core: Trunk-Based Development with Enhancements

```
main ← always green, always deployable, single source of truth
```

```
Each team works in short-lived branches (< 2 days):
  feat/payments-abc → main (via merge queue)
  fix/auth-xyz      → main (via merge queue)
  chore/upgrade-dep → main (direct on off-hours schedule)

Feature flags: incomplete features hidden behind flags
  → Engineers can commit to main daily without breaking users
  → Product controls rollout independently of code deploy
```

Merge Queue for Safety

```
No PR merges directly without queue:
  1. PR approved (CODEOWNERS review)
  2. CI passes on PR branch
  3. PR enters merge queue
  4. Merge queue: rebase PR onto current main + run CI
  5. If CI green → merge to main automatically
  6. If CI red → PR removed from queue, engineer notified

Result: main is NEVER broken by a merged PR
Throughput: with 5-PR batching, can process ~60 PRs/hour
```

Release Train for Teams With Monthly Cadence

```
For teams that can't move to daily releases:
  release/2024-Q1 ← feature freeze on Jan 31
                  ← bug fixes only after freeze
                  ← tagged and deployed Feb 7

Engineers on these teams work in main but their features
are behind flags until the release train includes them.
```

Hotfix Process

```
Production incident → hotfix branch from main (not from a release tag):
  1. git checkout -b hotfix/critical-fix main
  2. Fix, test
  3. Emergency PR → fast-track review (2 approvals required)
  4. Merge to main (bypasses queue with SRE override)
  5. Deploy immediately
  6. No backport needed: main IS production
```

Enforcement Automation

```
Branch protection on main:
  - 2 CODEOWNERS approvals required
  - All CI checks must pass
  - Merge queue required
  - Signed commits required
  - No force pushes, no direct pushes

CI requirements (must complete in < 10 min):
  - Unit tests for affected services only (monorepo change detection)
  - Integration tests for affected APIs
  - Security scan (gitleaks, dependency audit)
  - Build validation

Flag management:
  - LaunchDarkly / Unleash for feature flags
  - Automatic flag cleanup CI check (flags older than 90 days → required
review)

Observability:
  - Every merge to main → tagged with git SHA in monitoring
  - One-command rollback: git revert + push → auto-deploy
  - Mean time to rollback < 3 minutes
```

Short Interview Answer

> "For 200 engineers, I'd choose Trunk-Based Development with a merge queue as the safety mechanism. Short-lived branches (< 2 days) with feature flags for incomplete work keeps the feedback loop fast and avoids merge hell. The merge queue — GitHub Merge Queue or GitLab Merge Trains — ensures CI runs on the actual merged state, not just the PR branch, so main is never broken by a merge. CODEOWNERS with required reviews, signed commits, and branch protection rulesets provide governance. For teams with monthly release cadences, release trains with feature flags bridge the gap. The key metrics I'd track: time-to-merge (target < 4 hours), CI duration (target < 10 min), and main green percentage (target > 99.9%)."

🔗 How A1–A8 Connect to Each Other and to I1–I8

```
Git Internals Deep Dive (A1)
  → Explains WHY reflog works (objects persist after GC grace period)
  → Explains WHY force push is dangerous (SHA chain is broken for
everyone)
  → Explains HOW partial clones are possible (object store is separable)
  → Foundation for System Design (A8)

Advanced Rebase (A2)
  → --onto uses DAG merge-base computation (A1)
  → autosquash automates what I2 explained manually
  → feeds into Large Repo CI patterns (A3, A5)
```

```
Large Repo Strategies (A3)
  → Sparse checkout uses object filter from partial clone (A1)
  → LFS is an external object store (A1 pattern applied externally)
  → All gotchas surface in CI pipelines (A5)

Git Security (A4)
  → Signing adds identity on top of the SHA integrity chain (A1)
  → filter-repo rewrites the DAG, invalidating all downstream SHAs (A1,
A2)
  → CODEOWNERS enforcement connects to Protected Branches (A6)

CI/CD Pipelines (A5)
  → Shallow clones = intentionally incomplete DAG (A1)
  → git bisect = binary search on the DAG (A1)
  → Merge queues = serialized DAG extension (A1, A6)

Workflows at Scale (A6)
  → Stacked PRs = intentionally chained DAG paths (A1)
  → Merge queues build on CI/CD patterns (A5)
  → Fleet management uses everything from A1–A5

Troubleshooting and Recovery (A7)
  → Reflog explained deeply in A1
  → git fsck verifies SHA chain integrity (A1)
  → git bisect = DAG binary search (A1, A5)

System Design (A8)
  → Object storage design = A1 internals from first principles
  → Conflict detection = merge algorithm from I3 at depth
  → GitHub scaling = A5 CI patterns applied to infrastructure
  → Branching strategy = I5 applied at 200-engineer scale with A5/A6
tooling
```

📋 Quick Reference — Most Frequently Asked Advanced Questions

Question	One-Line Answer
What is a DAG in Git?	Commits pointing to parents: directed (backwards only), acyclic (no loops, SHA makes them impossible)
How does Git ensure integrity?	SHA chain: every object's hash includes its content and parent hashes — any change cascades through all descendant SHAs
What is the reflog?	Local journal of every HEAD/branch movement — your complete undo history for 90 days

Question	One-Line Answer
How do you recover a deleted branch?	`git reflog` → find last SHA → `git checkout -b branch SHA`
`git rebase --onto` usage?	`git rebase --onto newbase upstream branch` — transplants commits that are in branch but not upstream, onto newbase
What is a partial clone?	`--filter=blob:none`: downloads commits and trees, defers blob content until accessed
Sparse checkout vs partial clone?	Sparse = subset of working tree checked out; Partial = subset of objects downloaded
How do you remove a secret from Git history?	Rotate the secret first, then `git filter-repo --replace-text`, force push all branches, all teammates re-clone
What is a merge queue?	CI runs on PR + all queued PRs combined before any merges — prevents "tested separately, broken together"
How does `git bisect` work?	Binary search: marks commits good/bad, halves the search space each step — finds culprit in O(log N) steps
What is `git fsck`?	Verifies object connectivity and SHA integrity — diagnoses and finds corruption or orphaned objects
Why use `--force-with-lease` over `--force`?	`--force-with-lease` fails if remote has new commits since your last fetch, preventing teammate overwrites
What is trunk-based development?	Everyone commits to main directly or through <1-day branches + feature flags — eliminates merge hell
What is Scalar?	Git 2.38+ tool combining partial clone, sparse checkout, commit graph, and background maintenance for large repos
How does GitHub scale Git?	Consistent hashing for repo routing, Spokes replication (3+ copies), DGit distributed object store, read replicas
Author vs Committer in a commit?	Author = who wrote the code; Committer = who applied it — differ after cherry-pick or rebase of others' commits
What happens during git push end-to-end?	Auth → ref discovery → want/have negotiation → packfile transfer → server validation → atomic ref update → hooks

This completes the B1–B4 (Beginner), I1–I8 (Intermediate), and A1–A8 (Advanced) Git Interview Mastery series.